

Projektaufgabe 2

Phase 2 – Legen der Basis (2.5 P)

Datenmanagement jenseits von Relationen

Gruppen Nummer (A12)

Zanotti Christian, 12528509

Wiesinger Martin, 11935667

Ottino David, 51841010

May 5, 2026

Dieses Reporting Template dient der Vorbereitung der Abgabe von Phase 2.

Datengenerator für Matrizen mit Sparsity (0.5 Punkte)

Zeigen Sie den Code der Funktion `generate()` als Listing oder Screenshot. Gehen Sie (kurz) auf die wesentlichen Aspekte ein

Die Methode "generate" wird mit den Parametern `l` für die Dimension und `sparsity` für die Dichte aufgerufen. Es wird überprüft, ob `sparsity` im geforderten Bereich liegt und anschließend die Dimensionen (`m` und `n`) der beiden Matrizen anhand von `l` ermittelt. Anschließend werden zwei 2-D-Arrays erzeugt, indem über `l` und `m` bzw. `n` geloopt wird und in jedem Loop ein zufälliger Integer-Wert in das Array eingefügt wird. Mithilfe von `sparsity` werden auch an zufälligen Positionen 0-Werte eingefügt.

```
def generate(l: int, sparsity: float):  
  
    if not (0.0 <= sparsity <= 1.0):  
        raise ValueError("sparsity has to be between 0 and 1")  
  
    m = l - 1  
    n = l - 1  
  
    A = [  
        [random_value(sparsity) for _ in range(l)]  
        for _ in range(m)  
    ]  
  
    B = [  
        [random_value(sparsity) for _ in range(n)]  
        for _ in range(l)  
    ]  
  
    return A, B
```

```
def random_value(sparsity):
    if random.random() < sparsity:
        return 0.0
    else:
        return random.uniform(-10, 10)
```

Import der Matrizen in das DBMS (0.5 Punkte):

Geben Sie das Create Table Statement für Matrix A an.

```
CREATE TABLE IF NOT EXISTS A (
    i INT NOT NULL,
    j INT NOT NULL,
    val INT NOT NULL
);
```

Für die Übung bereiten Sie eine Demo des Datenimports vor.

Wahl des Toy Beispiels (0.5 P):

Geben Sie Matrix A und B als 2D Array an.

A:

```
[[1, 0, 2, 0],
 [0, 3, 0, 4],
 [5, 0, 0, 6]]
```

B:

```
[[7, 0, 8],
 [0, 9, 0],
 [1, 0, 2],
 [0, 3, 0]]
```

Zeigen Sie die äquivalente darstellung der Matrix A und B in der Datenbank

A:

"i"	"j"	"val"
1	1	1
1	3	2
2	2	3
2	4	4
3	1	5
3	4	6

B:

"i"	"j"	"val"
1	1	7
1	3	8
2	2	9
3	1	1
3	3	2
4	2	3

Implementierung von Ansatz 0 (0.5 P):

Zeigen Sie den Code der Matrixmultiplikation als Listing oder Screenshot. Erläutern Sie (kurz), welche Laufzeit ihr Algorithmus hat und warum das Kriterium keinen Algorithmus mit sub-kubischer Laufzeit zu wählen erfüllt ist

```
def ansatz0(A, B):
    m = len(A)
    l = len(A[0])
    n = len(B[0])

    C = [[0.0 for _ in range(n)] for _ in range(m)]

    for i in range(m):
        for j in range(n):
            for k in range(l):
                C[i][j] += A[i][k] * B[k][j]

    return C
```

Dem Algorithmus werden zwei 2-D-Arrays übergeben. Aus diesem werden die Parameter m , n und l ermittelt. Anschließend wird das Ergebnis-Array C in der richtigen Dimension initialisiert ($m \times n$). Die Berechnung selbst entspricht dem Standard-Schema für Matrix-Multiplikation: Jede Zeile in Matrix A wird mit jeder Spalte der Matrix B multipliziert. Die Summe dieser Multiplikation ist ein Ergebnis der Matrix C . Dieser Algorithmus hat kubische Laufzeit.

Implementierung von Ansatz 1 (0.5 P):

Berechnen Sie von Hand das Ergebnis $C = A \times B$ für ihr Toy Beispiel und geben Sie es nachfolgend an.

```
[[9.0, 0.0, 12.0],
 [0.0, 39.0, 0.0],
 [35.0, 18.0, 40.0]]
```

Zeigen Sie, dass Ihr System C korrekt berechnet (z.B. als Screenshot)

```
(1, 1, 9)
(1, 3, 12)
(2, 2, 39)
(3, 1, 35)
(3, 2, 18)
(3, 3, 40)
```

Zeitmanagement

Benötigte Zeit pro Person (nur Phase 1): 4

References

Important: Reference your information sources!

Remove this section if you use footnotes to reference your information sources.
